

Yes. If you show this **Flower Classification project** to an interview panel, they probably won't expect you to know every detail of TensorFlow. But they **will expect you to understand the parts you personally built** and explain why you made those choices.

For your current project, I would prepare around **6 areas**.

1. Machine Learning — most important

You should be able to explain:

- What is **image classification**?
- What are your **13 flower classes**?
- How did you collect/organize the dataset?
- Why did you split it into:
 - Training → 70%
 - Validation → 15%
 - Testing → 15%
- What is **stratification** and why did you use it?
- What is `label_index`?
- Why did you resize images to **224 × 224**?
- Why normalize pixels using `/255.0`?
- What is **data augmentation**?
- Why use:
 - `RandomFlip`
 - `RandomRotation`
 - `RandomZoom`

You should be able to explain the entire pipeline:

Images → preprocessing → augmentation → MobileNetV2 → training → evaluation → saved model

2. MobileNetV2 / Transfer Learning

This is a particularly good interview topic from your project.

You should know:

What is MobileNetV2?

A lightweight CNN architecture designed to work efficiently, especially on resource-constrained devices.

Why did you use it?

Because training a CNN completely from scratch would require considerably more data and computational resources. MobileNetV2 already learned useful visual features from ImageNet, so you used **transfer learning**.

Your code:

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)
```

```
base_model.trainable = False
```

You should be able to say:

"I removed MobileNetV2's original classification head, kept its pretrained feature extractor frozen, and added my own classification layers for the 13 flower classes."

That's a **very good interview answer**.

3. Your model architecture

Know what these layers do:

Data Augmentation



MobileNetV2



GlobalAveragePooling2D



Dropout



Dense(13, softmax)

Especially understand:

GlobalAveragePooling2D

It converts the spatial feature maps into a compact feature vector before classification.

Dropout(0.2)

Randomly disables some neurons during training to reduce overfitting.

Dense(13, softmax)

Because you have **13 mutually exclusive flower classes**.

Softmax gives probabilities such as:

Golap	0.954
Zinnia	0.018
Jaba	0.009
...	

The highest probability becomes the prediction.

4. Your results

This is something you **absolutely should know**.

Your current result was:

Training accuracy	≈ 97.2%
Validation accuracy	≈ 96.7%
Test accuracy	≈ 95.25%

So if they ask:

"How well does your model perform?"

You can answer:

"After five epochs, my model achieved approximately 95.25% accuracy on the unseen test dataset. The validation accuracy was around 96.66%."

And if they ask:

"Why isn't test accuracy 100%?"

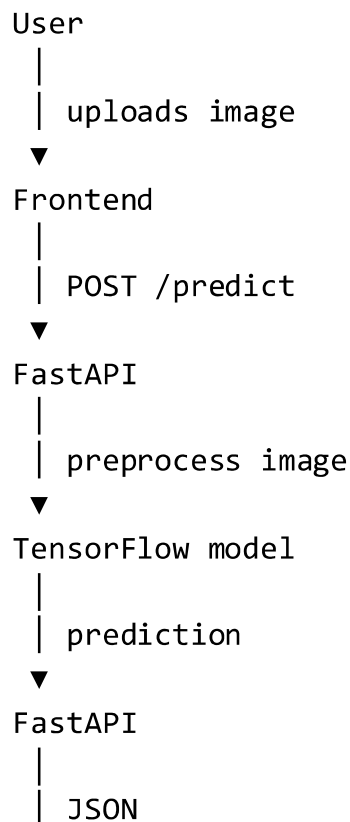
Say:

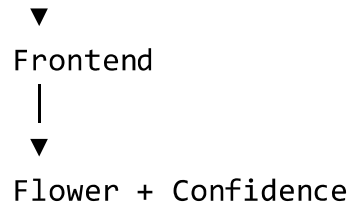
"Because the model hasn't seen the test images during training, and image classification naturally has some difficult or visually similar samples. The test set provides a more realistic estimate of generalization."

5. FastAPI + website

This is another area where they can ask questions because you have actually turned the ML model into an application.

Understand this architecture:





You should know:

What is FastAPI?

A Python framework for building APIs.

Why did you use it?

Because your ML model is written in Python/TensorFlow, so FastAPI provides a convenient way to expose the model through an HTTP API.

What does `/predict` do?

It accepts the uploaded image, preprocesses it to the format expected by the model, performs prediction, and returns the flower name and confidence.

For example:

```
{  
  "flower": "Golap",  
  "confidence": 95.42  
}
```

6. Frontend

They may ask:

"How does your website communicate with FastAPI?"

Your answer:

"The JavaScript frontend uses the Fetch API to send the selected image as multipart form data to the `/predict` endpoint. FastAPI processes the image and returns JSON, which JavaScript uses to update the prediction result on the page."

That's an important concept.

Questions I would EXPECT them to ask you

For your particular project, prepare these very well:

Interview question	Importance
What problem does your project solve?	★★★★★
Explain your project architecture	★★★★★
Why MobileNetV2?	★★★★★
What is transfer learning?	★★★★★
Why 224×224?	★★★★
Why /255.0?	★★★★
Why train/validation/test split?	★★★★★
What is stratification?	★★★★
What is data augmentation?	★★★★
Why freeze MobileNetV2?	★★★★★
Why softmax?	★★★★
Why 13 neurons in Dense?	★★★★
What does dropout do?	★★★★
What is an epoch?	★★★★
What is batch size?	★★★
What is overfitting?	★★★★★
What is accuracy?	★★★★
Why test accuracy differs from training accuracy?	★★★★
What is FastAPI?	★★★★★
What is an API?	★★★★★
What does /predict do?	★★★★★
How does frontend communicate with backend?	★★★★★
Why save .keras?	★★★★
Why save class_names.npy?	★★★★

How would you deploy this?



How would you improve it?



And there is one VERY important thing

Don't try to memorize the code line-by-line.

An interviewer may point to this:

```
train_data, temporary_data = train_test_split(
    data,
    test_size=0.30,
    stratify=data["label_index"],
    random_state=SEED
)
```

and ask:

"Explain this."

You should be able to explain it naturally:

"I'm splitting the dataset into 70% training and 30% temporary data. I'm using stratification so that the proportion of each flower class remains similar in the splits, and `random_state=42` makes the split reproducible."

That's much more valuable than memorizing the code.

One thing I would improve before an interview

Your project currently has a **very good first version**, but don't present it as merely:

"I made a website that identifies flowers."

Present it as:

"I developed an end-to-end image-classification system using transfer learning with MobileNetV2. I prepared and stratified a 13-class dataset, performed preprocessing and augmentation, trained and evaluated the model, saved the trained model, exposed it through a FastAPI REST endpoint, and built a frontend that sends images to the API and displays the prediction and confidence."

That sounds substantially more professional because it demonstrates that you understand the **complete ML-to-application pipeline**, not just a TensorFlow tutorial.

And if they ask **"Did you build this yourself?"**, you should be able to explain every major decision in that paragraph.

If you want, I can also take **your exact current project** and conduct a **real interview simulation**, asking you questions one-by-one exactly like an interviewer and judging your answers.